User defined data types in C:

The UDT (User-Defined Data Type) is a typical data type that we can derive out of any existing data type in a program. We can utilize them for extending those built-in types that are already available in a program, and then you can create various customized data types of your own.

Types of User-Defined Data Types in C:

1. Structure
2. Union
3. enumeration

1)Structure in C:

- Structure is a user defined data type which hold or store heterogeneous data item or element in a singe variable.
- It is a Combination of primitive and derived data type.
- The structure is a collection of dissimilar elements in contiguous memory locations, under a single name. They are user-defined data types.
- The name of the structure (or the tag name) is treated as a data type, and the elements of the structure are known as its members.
- No memory is allocated during the definition of the structure.
- Memory is only allocated when its variables are created (which is usually preceded by the keyword struct).
- The variables of the structure types occupy the memory size which is equal to the sum of the size of all its members.
- At one time simultaneously data can be stored in all its members.
- The members can be accessed by using the dot (.) operator.
- All primitive data types are used to manipulate basic datatypes i.e. char, int, float. All derived datatypes work for primitive datatypes.
- In the real-world, every information will be there in the form of objects.
- Every object having its own properties and behavior. No, any primitive or derived datatypes support real-time object information.
- When the primitive or derived datatypes are not supporting user requirement then go for user-defined datatypes.

Some important point about structure:

- A Structure is a collection of different types of data elements in a single entity.
- A Structure is a combination of primitive and derived datatype variables.
- By using structures, we can create user-defined datatypes.
- The size of the structure is the sum of all member variable sizes.
- The least size of structure is 1byte.
- In 'C' programming language, it is not possible to create an empty structure.
- C language structure contains data members only but in C++, data members, and member functions.

Use of Structure in C:

In C language array is also a user defined data type but array hold or store only similar type of data, If we want to store different-different type of data in then we need to defined separate variable for each type of data.

When use Structure?: When need to manipulate the data for all member variables then use structure

Example: **Suppose we want to store Student record, then we need to store....**

- Student Name
- Roll number
- Class
- Address

For store Student name and Address we need character data type, for Roll number and class we need integer data type.

If we are using Array then we need to defined separate variable.

Example:

```
char  studentname[10], address[20];
int   rollno[5], class[5];
```

If we use Structure then we use single variable for all data.

Syntax:

```
struct  stu
{
char  studentname[10];
char  address[20];
int   rollno[5];
```

int class[5];
};

Note: Minimum size of Structure is one byte and Maximum size of Structure is sum of all members variable size.

Note: Empty Structure is not possible in C Language.

Defining a Structure:

Syntax:

```
struct tagname
{
datatype-1 member-1;
datatype-2 member-2;
datatype-3 member-3;
.....;
.....;
datatype-n member-n;
};
```

At end of the structure creation (;) must be required because it indicates that an entity is constructed.

Example:

```
struct emp
{
int id;
char name[36];
int sal;
};
```

sizeof(struct emp) // --> 40 byte (2byte+36byte+2byte)

Note: In case of structure all data members initialize at a time because memory location are different

memory allocation of Structure:

Syntax:

```
struct emp
{
int ID;
char name[10];
double salary;
};
```

For above structure, memory allocation like below.

int ID – 2 Bytes

char name[10] – 10 Bytes

double salary – 8 Bytes

Total memory allocation=2+10+8=20 Bytes

Syntax to create structure variable:

```
struct tagname variablename;
```

Example:

```
struct emp e;
```

in the above example **emp** is a structure tagname and **e** is name of a structure variable

Syntax to access structure members:

By using following operators we can access structure members.

Syntax:

. struct to member

--> pointer to member

- When the variable is normal type then go for struct to member selection(dot)operator.
- When the variable is pointer type then go for pointer to member (arrow)operator.

Example program1:

```
#include<stdio.h>
#include<conio.h>
struct Student
{
  char sname[30];
  int rno;
```

```
   float   avg;
} ;
void  main()
{
   struct   Student   s={“Rahul”,10,89.6};
   clrscr();
   printf("\n studnt name=%s”,s.sname);
   printf(“\n student roll number=%d”,s.rno);
  printf(“\n student average=%f”,s.avg);
getch();
}
```

Output:

```
  studnt name=rahul
  student roll number=10
  student average=89.6
```

Example program2:

```
#include<stdio.h>
#include<conio.h>
struct   Rectangle
{
   int   length;
   int   breadth;
};
void   main()
{
   struct   Rectangle   r={ 10, 5 };
   clrscr();
   printf ("length:%d", r.length);
   printf ("breadth:%d",r.breadth);
   printf ("Area of Rectangle: %d", r.length*r.breadth);
   getch();
}
```

Output:

```
Length:10
Breadth:5
Area of Rectangle:50
```

Example program3:

```
#include<stdio.h>
#include<conio.h>
struct   emp
{
int id;
char name[36];
float sal;
};
void main()
{
struct   emp   e;
clrscr();
printf("Enter employee Id, Name, Salary: ");
scanf("%d %s %f",&e.id,&e.name,&e.sal);
printf("Employee Id: %d",e.id);
printf("\nEmployee Name: %s",e.name);
printf("\nEmployee Salary: %f",e.sal);
getch();
}
```

Output: Enter employee Id, Name, Salary: 5

```
 jaiswal
 45000
Employee Id :5
Employee  Name: jaiswal
Employee  Salary: 45000.00
```

Example program4:

```
#include<stdio.h>
#include<conio.h>
struct   Student
{
   char   stud_name[30];
   int     roll_number;
   float   percentage;
} ;
void  main()
{
   struct   Student   stud_1;  // using struct keyword
   clrscr();
   printf("Enter details of stud_1 : \n");
   printf("Name : ");
   scanf("%s", stud_1.stud_name);
   printf("Roll Number : ");
   scanf("%d", &stud_1.roll_number);
   printf("Percentage : ");
   scanf("%f", &stud_1.percentage);
   printf("***** Student 1 Details *****\n);
   printf("Name of the Student : %s\n", stud_1.stud_name);
   printf("Roll Number of the Student : %i\n", stud_1.roll_number);
   printf("Percentage of the Student : %f\n", stud_1.percentage);
   getch();
}
```

2)Union in C:

- A union is a collection of different types of data elements in a single entity.
- It is a collection of primitive and derived datatype variables.
- By using a union, we can create user-defined data type elements.
- The size of a union is max size of a member variable.
- In the implementation, for manipulation of the data, if we are using only one member then it is recommended to go for the union.
- When we are working with unions, all member variables will share the same memory location.
- By using union, when we are manipulating multiple members then actual data is lost.
- The union is also a collection of dissimilar elements in contiguous memory locations, under a single name.
- They are user-defined data types. The name of the union (or the tag name) is treated as a data type, and the elements of the structure are known as its members.
- No memory is allocated during the definition of the union. Memory is only allocated when its variables are created (which is usually preceded by the keyword union).
- The variables of the union types occupy the memory size which is the maximum size among all its members.
- At one time simultaneously data can be stored in only one of its members.
- The members can be accessed by using the. (I.e. dot) operator.
- The union is quite similar to the structures in C. The union is also a derived type of structure.
- A union can be defined in the same manner as structures just the keyword used in defining union in the union where keyword used in defining structure was struct.

Defining a union:

Union can be defined in same manner as structures, for defining union use union keyword where as for defining structure use struct keyword.

Syntax:

```
union   tagname
{
datatype-1    member-1;
datatype-2    member-2;
```

…….
…….
datatype-n member-n;
};

Example:

```
union emp
{
int ID;
char name[10];
double salary;
}u;
```

Accessing members of an union:

The member of unions can be accessed in similar manner as Structure with union reference. Suppose, we you want to access name variable in above example, it can be accessed as u.name.

Advantage of union over structure:

- It occupies less memory because it occupies the memory of largest member only.

Disadvantage of union over structure:

- It can store data in one member only.

Memory Allocation in Union:

Union allocates one common storage space for its all members. Union find which member need more memory than other member, then it allocate that much space

Syntax:

```
union emp
{
int ID;
char name[10];
double salary;
};
```

For above union, only 8 bytes of memory will be allocated because double data type will occupy maximum space of memory over other data types.

Total memory allocation = 8 Bytes

When use Union:

- When need to manipulate only one member then use union.

Note:

- All the properties of the union are same for structure, except initialization process.
- in case of union only one member need to be initialize.
- In case of union if we initializing multiple member then compiler will gives an error.

Example Program:

```
#include<stdio.h>
#include<conio.h>
union alpha
{
  int i;
  float f;
};
void main()
{
  union alpha a;
  clrscr();
  a.i=10;
  printf ("\n i=%d", a.i);
  a.f = 220.5;
  printf ("\n f= %f", a.f);
  getch();
}
```

Output: i=10
f=220.5

Example program2:

```
#include<stdio.h>
#include<conio.h>
union   emp
{
int   ID;
char   name[10];
double   salary;
}u; //  reference of union
void main()
{
clrscr();
printf("Enter emp Id: ");
scanf("%d",&u.ID);
printf("Enter emp Name: ");
scanf("%s",&u.name);
printf("Enter emp Salary: ");
scanf("%f",&u.salary);
printf("Emp ID: %d",u.ID);
printf("Emp Name: %s",u.name);
printf("Emp Salary: %f",u.salary);
getch();
}
```

Output: Emp ID: 100
Emp Name: Porter
Emp Salary: 20000

Example program3:

```
union   Student
{
  char stud_name[30];
  int roll_number;
  float percentage;
} ;
void   main()
{
  union   Student   stud_1;
  clrscr();
  printf("Enter details of stud_1 : \n");
  printf("Name : ");
  scanf("%s", stud_1.stud_name);
  printf("Roll Number : ");
  scanf("%d", &stud_1.roll_number);
  printf("Percentage : ");
  scanf("%f", &stud_1.percentage);
  printf("***** Student 1 Details *****\n);
  printf("Name of the Student : %s\n", stud_1.stud_name);
  printf("Roll Number of the Student : %i\n", stud_1.roll_number);
  printf("Percentage of the Student : %f\n", stud_1.percentage);
  getch();
}
```

In the above example program, the union variable "stud_1 is created while defining the union. Whenever we access the members of a union we use the dot (.) operator.

Comparative advantages and disadvantages of the structure:

Below is the list of some advantages and disadvantages of using structure:

Advantages of Structure:

- Structure stores more than one data type of the same object together.
- It is helpful when you want to store data of different or similar data types such as name, address, phone, etc., of the same object.

- It makes it very easy to maintain the entire record as we represent complete records using a single name.
- The structure allows passing a whole set of records to any function with the help of a single parameter.
- An array of structures can also be created to store multiple data of similar data types.

Disadvantages of Structure:

- If the complexity of the project goes increases, it becomes hard to manage all your data members.
- Making changes to one data structure in a program makes it necessary to change at several other places. So it becomes difficult to track all changes.
- The structure requires more storage space as it allocates memory to all the data members, and even it is slower.
- The structure takes more storage space as it gives memory to all the different data members, whereas union takes only the memory size required by the largest data size parameters, and the same memory is shared with other data members.

Comparative advantages and disadvantages of union:

Below is the list of some advantages and disadvantages of using union:

Advantages of Union:

- Union takes less memory space as compared to the structure.
- Only the largest size data member can be directly accessed while using a union.
- It is used when you want to use less (same) memory for different data members.
- It allocates memory size to all its data members to the size of its largest data member.

Disadvantages of Union:

- It allows access to only one data member at a time.
- Union allocates one single common memory space to all its data members, which are shared between all of them.
- Not all the union data members are initialized, and they are used by interchanging values at a time.

Similarities between Structure and Union:

- Both are user-defined data types used to store data of different types as a single unit.
- Their members can be objects of any type, including other structures and unions or arrays. A member can also consist of a bit field.
- Both structures and unions support only assignment = and sizeof operators. The two structures or unions in the assignment must have the same members and member types.
- A structure or a union can be passed by value to functions and returned by value by functions. The argument must have the same type as the function parameter. A structure or union is passed by value just like a scalar variable as a corresponding parameter.
- '.' operator is used for accessing members.

Difference between Structure and Union:

Both a structure and a union combine different information related to a given entity. The main difference between the two is how specific information is stored and accessed.

The table below highlights the key differences between structure and union:

Sno	Structure	Union
1	A user can deploy the keyword **struct** to define a Structure.	A user can deploy the keyword **union** to define a Union.
2	The implementation of Structure in C occurs internally-because it contains separate memory locations allotted to every input member.	In the case of a Union, the memory allocation occurs for only one member with the largest size among all the input variables. It shares the same location among all these members/objects.
3	A user can access individual members at a given time.	A user can access only one member at a given time.

4	**The Syntax of declaring a Structure in C is:**
	struct structurename
	{
	datatype-1 member-1;
	datatype-2 member-2;
	.
	.
	datatype-n member-n;
	};
		The Syntax of declaring a Union in C is:
		union unionname
		{
		datatype-1 member-1;
		datatype-2 member-2;
		.
		.
		datatype-n member-n;
		};

5	A Structure does not have a shared location for all of its members. It makes the size of a Structure to be greater than or equal to the sum of the size of its data members.	A Union does not have a separate location for every member in it. It makes its size equal to the size of the largest member among all the data members.
6	Altering the values of a single member does not affect the other members of a Structure.	When you alter the values of a single member, it affects the values of other members.
7	In the case of a Structure, there is a specific memory location for every input data member. Thus, it can store multiple values of the various members.	In the case of a Union, there is an allocation of only one shared memory for all the input data members. Thus, it stores one value at a time for all of its members.

8
 	
9
	In the case of a Structure, a user can initialize multiple members at the same time.
	Example program:
	#include<stdio.h>
	#include<conio.h>
	struct Student
	{
	char sname[30];
	int rno;
	float avg;
	};
	void main()
	{
	struct Student s={"Rahul",10,89.6};
	clrscr();
	printf("\n studnt name=%s",s.sname);
	printf("\n student roll number=%d",s.rno);
	printf("\n student average=%f",s.avg);
	getch();
	}
	Output:
	studnt name=rahul
	student roll number=10
	student average=89.6
		In the case of a Union, a user can only initiate the first member at a time.
		Example Program:
		#include<stdio.h>
		#include<conio.h>
		union alpha
		{
		int i;
		float f;
		};
		void main()
		{
		union alpha a;
		clrscr();
		a.i=10;
		printf ("\n i=%d", a.i);
		a.f = 220.5;
		printf ("\n f=%f", a.f);
		getch();
		}
		Output:
		i=10
		f=220.5

(Or)

Difference between Structure and Union:

Both a structure and a union combine different information related to a given entity. The main difference between the two is how specific information is stored and accessed. The table below highlights the key differences between structure and union:

	Structure	Union
1.	We use the struct keyword to define a structure.	We use the union keyword to define a union.
2.	Every member is assigned a unique memory location.	All the data members share a single memory location.

3.	Change in the value of one data member does not affect the the value value of other data members in the structure.	Change in the value of one data member affects of other data member
4.	You can initialize multiple members at a time.	You can initialize only the first member at once.
5.	A structure can store multiple values of the different members.	A union stores one value at a time for all of its members.
6.	Structure's total size is the sum of the size of every data member. data member.	A union's total size is the size of the largest
7.	Users can access or retrieve any member at a time. a time.	You can access or retrieve only one member at

3) Enumeration in C:

An enum is a keyword, it is an user defined data type. All properties of integer are applied on Enumeration data type so size of the enumerator data type is 2 byte. It work like the Integer.

It is used for creating an user defined data type of integer. Using enum we can create sequence of integer constant value.

Syntax:

enum tagname{value1, value2, value3,....};

- In above syntax enum is a keyword. It is a user defiend data type.
- In above syntax tagname is our own variable. tagname is any variable name.
- value1, value2, value3,.... are create set of enum values.

It is start with 0 (zero) by default and value is incremented by 1 for the sequential identifiers in the list. If constant one value is not initialized then by default sequence will be start from zero and next to generated value should be previous constant value one.

Example:

enum week {sun, mon, tue, wed, thu, fri, sat};
enum week today;

- In above code first line is create user defined data type called week.
- week variable have 7 value which is inside { } braces.
- today variable is declare as week type which can be initialize any data or value among 7 (sun, mon,....).

Example program:

```
#include<stdio.h>
#include<conio.h>
enum ABC{x,y,z};
void main()
{
int a;
clrscr();
a=x+y+z;  //0+1+2
printf("Sum: %d",a);
getch();
}
```

Output:

Sum: 3

Example program2:

```
#include<stdio.h>
#include<conio.h>
enum week{sun, mon, tue, wed, thu, fri, sat};
void main()
{
 enum week today;
 clrscr();
 today=tue;
 printf("day=%d",today+1);
 getch();
}
```

Output:

day=3

Here "enum week" is user defined data type and today is integer type variable which initialize Tuesday. In above code we add "today+1" because enum is start from 0, so to get proper answer we add "1". Other wise it give "day=2".

Example program3:

```
#include<stdio.h>
#include<conio.h>
enum    week{sun, mon, tue, wed, thu, fri, sat};
void   main()
{
for(i=sun; i<=sat; i++)
{
 printf("%d ",i);
}
getch();
}
```

Output:

0 1 2 3 4 5 6

In above code replace sun, mon, tue,.... with Equivalent numeric value 0, 1, 2,...

Example Program4 for enum with default values:

```
#include<stdio.h>
#include<conio.h>
enum    day{ Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday} ;
void main()
{
  enum  day  today;
  clrscr();
  today = tuesday ;
   printf("\ntoday = %d ", today) ;
   getch();
}
```

In the above example program a user defined datatype "day is created seven values, monday as integral constant '0', tuesday as integral constant '1', wednesday as integral constant '2', thursday as integral constant '3', friday as integral constant '4', saturday as integral constant '5' and sunday as integral constant '6'. Here, when we display tuesday it displays the respective integral constant '1'.

We can also change the order of integral constants, consider the following example program.

Example Program5 for enum with changed integral constant values:

```
#include<stdio.h>
#include<conio.h>
enum    day{ Monday = 1, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday} ;
void main()
{
  enum   day   today;
  clrscr();
  today = tuesday ;
  printf("\ntoday = %d ", today) ;
  getch();
}
```

In the above example program, the integral constant value starts with '1' instead of '0'. Here, tuesday value is displayed as '2'.

We can also create enum with our own integral constants, consider the following example program.

Example Program6 for enum with defferent integral constant values:

```
#include<stdio.h>
#include<conio.h>
enum    hold{low = 40, normal = 60, high = 100} ;
void  main()
{
  enum   hold  status;
  clrscr();
  status = low ;
   printf("\nYou are at %d state. Work hard to improve!!", status) ;
   getch();
}
```

Some times we may assign our own integral constant from other than the first name. In this case, compiler follows default integral constants for the name that is before the user-defined integral constant and from user-defined constant onwards it resumes. Consider the following example program.

Example Program7 for enum with changed integral constant values:

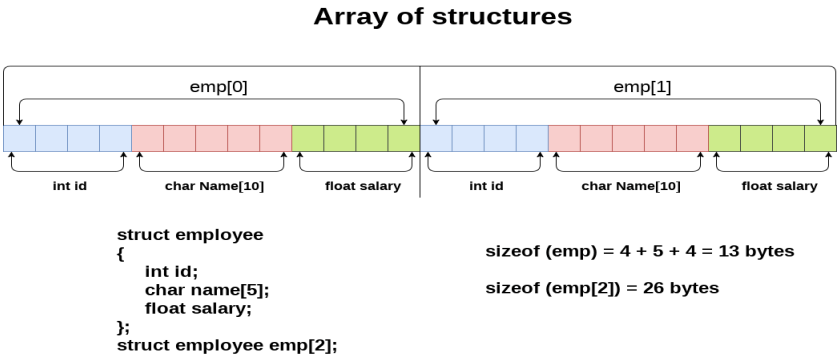
```
#include<stdio.h>
#include<conio.h>
enum    day{Monday, Tuesday, Wednesday, Thursday = 10, Friday, Saturday, Sunday} ;
void main()
{
  enum   day   today;
  clrscr();
  today=tuesday ;
   printf("\ntoday = %d ", today) ;
   today = saturday ;
   printf("\ntoday = %d ", today) ;
   getch();
}
```

In the above example program a user defined datatype "day is created seven values, monday as integral constant '0', tuesday as integral constant '1', wednesday as integral constant '2', thursday as integral constant '10', friday as integral constant '11', saturday as integral constant '12' and sunday as integral constant '13'.

Note - In enumeration, more than one name may given with same integral constant.

Array of Structures in C:

An array of structres in C can be defined as the collection of multiple structures variables where each variable contains information about different entities. The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.



Example program:

```
#include<stdio.h>
#include <conio.h>
struct   student
{
int   rollno;
char name[10];
float avg;
};
void    main()
{
int i;
struct   student   st[2]={10,"Rahul",78.2,11,"suraj",86.9};
clrscr();
for(i=0;i<2;i++)
{
printf("\n %d \t %s \t %f",st[i].rollno,st[i].name,st[i].avg);
}
getch();
}
```

Output:

```
10  rahul   78.2
11  suraj   86.9
```

Example program2: Let's see an example of an array of structures that stores information of 5 students and prints it.

```
#include<stdio.h>
#include <conio.h>
struct  student
{
int  rollno;
char name[10];
};
void   main()
{
int i;
struct  student  st[5];
printf("Enter Records of 5 students");
for(i=0;i<5;i++)
{
printf("\nEnter Rollno:");
scanf("%d",&st[i].rollno);
printf("\nEnter Name:");
scanf("%s",&st[i].name);
}
printf("\nStudent Information List:");
for(i=0;i<5;i++)
{
printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
 getch();
}
```

Output:

```
Enter Records of 5 students
Enter Rollno:1
Enter Name:Sonoo
Enter Rollno:2
Enter Name:Ratan
Enter Rollno:3
Enter Name:Vimal
Enter Rollno:4
Enter Name:James
Enter Rollno:5
Enter Name:Sarfraz
Student Information List:
Rollno:1, Name:Sonoo
Rollno:2, Name:Ratan
Rollno:3, Name:Vimal
Rollno:4, Name:James
Rollno:5, Name:Sarfraz
```

Nested Structure in C:

It is a procedure of placing a structure within an existing structure body. When we are working with a nested structure, the size of the structure is the sum of inner structure properties and outer structure properties are to be calculated. When we are working with a nested structure, it is not possible to access inner structure members directly by using the outer structure variable. In order to access inner structure members by using the outer structure variable, we are required to create inner structure variable within the body only. The inner structure variable does not have to access outer structure members directly or indirectly. Structures can be nested within other structures in C programming.

Example program:

```
#include<stdio.h>
#include <conio.h>
```

```
struct   student
{
int   rollno;
char name[10];
  struct marks
 {
    m1,m2,m3;
 }m;
};
void main()
{
Struct student s={10,”Rahul”,89,78,90};
Clrscr();
printf("\n Student rollno=%d”,s.rno);
printf(“\n Student name= %s”,s.name);
printf(\n marks1=%d",s.m.m1);
printf(\n marks2=%d",s.m.m2);
printf(\n marks3=%d",s.m.m3);
getch();
}
```

Output:

```
Student rollno=10
Student name=rahul
marks1=89
marks2=78
marks3=90
```

Exampleprogram2:

```
#include <stdio.h>
#include<conio.h>
#include <string.h>
struct   Employee
{
   int id;
   char name[20];
   struct    Date
   {
      int dd;
      int mm;
      int yyyy;
   } doj;
} e1;
void   main()
{
    clrscr();
   //storing employee information
   e1.id = 101;
   e1.name="Krish";
   e1.doj.dd = 10;
   e1.doj.mm = 11;
   e1.doj.yyyy = 2014;
   //printing first employee information
   printf ("employee id : %d\n", e1.id);
   printf ("employee name : %s\n", e1.name);
   printf ("employee date of joining (dd/mm/yyyy) : %d/%d/%d\n", e1.doj.dd, e1.doj.mm, e1.doj.yyyy);
   getch();
}
```

Output:

```
employee id : 101
employee name : Krish
employee date of joining (dd/mm/yyyy) : 10/11/2014
```

Example program3:

```
#include<stdio.h>
#include<conio.h>
struct  address
{
  char city[20];
  int pin;
  char phone[14];
};
struct  employee
{
  char name[20];
  struct address  add;
};
void main ()
{
  struct  employee  emp;
  clrscr();
  printf("Enter employee information?\n");
  scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.add.phone);
  printf("Printing the employee information....\n");
  printf("name: %s\nCity: %s\nPincode: %d\nPhone: %s",emp.name,emp.add.city,emp.add.pin,emp.add.phone);
  getch();
}
```

Output: Enter employee information?
Arun
Delhi
110001
1234567890
Printing the employee information....
name: Arun
City: Delhi
Pincode: 110001
Phone: 1234567890

Structure and Pointer:

Pointers can be accessed along with structures. Structure's member through a pointer can be used in two ways:

- Referencing pointer to another address to access memory.
- Using dynamic memory allocation.

Example:

```
#include <stdio.h>
#include<conio.h>
#include <string.h>
struct  student
{
  int id;
  char name[30];
  float percentage;
};
void  main()
{
  int  i;
  struct  student  record1={ 1, "Raju", 90.5 };
```

```
  struct  student  *ptr;
  clrscr();
  ptr=&record1;
  printf ("Records of STUDENT1: \n");
  printf ("Id is: %d \n", ptr->id);
  printf ("Name is: %s \n", ptr->name);
  printf ("Percentage is: %f \n\n", ptr->percentage);
  getch();
}
```

Output:

```
Records of STUDENT1:
Id is: 1
Name is: Raju
Percentage is: 90.500000
```

Structure pointer member can also be accessed using -> operator.

- (*ptr).a is same as ptr->a
- (*ptr).b is same as ptr->b

Files in C:

Generally, a file is used to store user data in a computer. In other words, computer stores the data using files. we can define a file as follows... File is a collection of data that stored on secondary memory like haddisk of a computer.

It is necessary to keep data in the permanent storage because it is difficult to handle the large volume of data by programs and after execution of program is over all the entered data will be lost because, the data stored in the variables are temporary.

C supports the concept of file through which the data can be stored in the disk or secondary storage device. So the data can be read when required. A file is a collection of data or text, placed on the disk.

Hence depending up on the method of accessing the data stored ,there are two types of files.

1. Sequential file
2. Random access file

1. Sequential File: In this type of files data is kept in sequential order if we want to read the last record of the file,we need to read all records before that record so it takes more time.

2. Random access Files: In this type of files data can be read and modified randomly .If we want to read the last record we can read it directly.It takes less time when compared to sequential file.

Generally, a file is used to store user data in a computer. In other words, computer stores the data using files. we can define a file as follows... File is a collection of data that stored on secondary memory like haddisk of a computer.

File Operations in C:

The following are the operations performed on files in c programming language:

- Creating (or) Opening a file
- Reading data from a file
- Writing data into a file
- Closing a file

All the above operations are performed using file handling functions available in C.

C programming language supports two types of files and they are as follows:

1. Text Files (or) ASCII Files
2. Binary Files

1.Text File (or) ASCII File:The file that contains ASCII codes of data like digits, alphabets and symbols is called text file (or) ASCII file.

A Text file contains only the text information like alphabets ,digits and special symbols. The ASCII code of these characters are stored in these files.It uses only 7 bits allowing 8 bit to be zero.

1.w(write):

This mode opens new file on the disk for writing.If the file exist,disk for writing.If the file exist, then it will be over written without then it will be over written without any confirmation.

SYNTAX:

1 fp=fopen("data.txt","w");

2 "data.txt" is filename

3 "w" is writemode.

2.r(read):
This mode opens an preexisting file for reading.If the file doesn't Exist then the compiler returns a NULL to the file pointer
SYNTAX:
fp=fopen("data.txt","r");
3. w+(read and write):
This mode searches for a file if it is found contents are destroyed If the file doesn't found a new file is created.
SYNTAX:
fp=fopen("data.txt","w+");
4.a(append):
This mode opens a preexisting file for appending the data.
SYNTAX:
fp=fopen("data.txt","a");
5.a+(append+read):
the end of the file.
SYNTAX:
fp=fopen("data.txt","a+");
6.r+ (read +write):
This mode is used for both Reading and writing

2.Binary File: The file that contains data in the form of bytes (0's and 1's) is called as binary file. Generally, the binary files are compiled version of text files.
A binary file is a file that uses all 8 bits of a byte for storing the information .It is the form which can be interpreted and understood by the computer.
The only difference between the text file and binary file is the data contain in text file can be recognized by the word processor while binary file data can't be recognized by a word processor.

1.wb(write):
this opens a binary file in write mode.
SYNTAX:
fp=fopen("data.dat","wb");
2.rb(read):
this opens a binary file in read mode
SYNTAX:
fp=fopen("data.dat","rb");
3.ab(append):
this opens a binary file in a Append mode i.e. data can be added at the end of file.
SYNTAX:
fp=fopen("data.dat","ab");
4.r+b(read+write):
this mode opens preexisting File in read and write mode.
SYNTAX:
fp=fopen("data.dat","r+b");
5.w+b(write+read):
this mode creates a new file for reading and writing in Binary mode.
SYNTAX:
fp=fopen("data.dat","w+b");
6.a+b(append+write):
this mode opens a file in append mode i.e. data can be written at the end of file.
SYNTAX:
fp=fopen("data.dat","a+b");

File Handling (or)file management Functions in C:

File is a collection of data that stored on secondary memory like hard disk of a computer.
The following are the operations performed on files in the c programming language...

1. Creating (or) Opening a file
2. Reading data from a file
3. Writing data into a file
4. Closing a file

All the above operations are performed using file handling functions available in C.

1)Creating (or) Opening a file:

To create a new file or open an existing file, we need to create a file pointer of FILE type. Following is the sample code for creating file pointer.

File *f_ptr ;

We use the pre-defined method fopen() to create a new file or to open an existing file. There are different modes in which a file can be opened. Consider the following code...

File *f_ptr ;

*f_ptr = fopen("abc.txt", "w") ;

The above example code creates a new file called abc.txt if it does not exists otherwise it is opened in writing mode.

In C programming language, there different modes are available to open a file and they are shown in the following table.

S. No.	Mode	Description
1	r	Opens a text file in reading mode.
2	w	Opens a text file in wirting mode.
3	a	Opens a text file in append mode.
4	r+	Opens a text file in both reading and writing mode.
5	w+	Opens a text file in both reading and writing mode. It set the cursor position to the begining of the file if it exists.
6	a+	Opens a text file in both reading and writing mode. The reading operation is performed from begining and writing operation is performed at the end of the file.

Note - The above modes are used with text files only. If we want to work with binary files we use rb, wb, ab, rb+, wb+ and ab+.

2)Reading from a file:

The reading from a file operation is performed using the following pre-defined file handling methods.

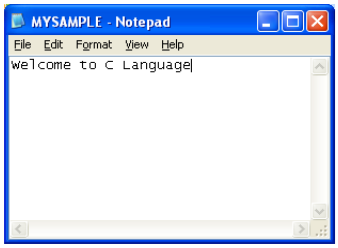
a) getc()
b) getw()
c) fscanf()
d) fgets()
e) fread()

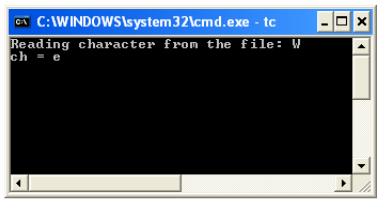
a)getc(*file_pointer):This function is used to read a character from specified file which is opened in reading mode. It reads from the current position of the cursor. After reading the character the cursor will be at next character.

Example Program to illustrate getc() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
  FILE  *fp;
  char   ch;
  clrscr();
  fp = fopen("MySample.txt","r");
  printf("Reading character from the file: %c\n",getc(fp));
  ch = getc(fp);
  printf("ch = %c", ch);
  fclose(fp);
  getch();
  }
```

Output:





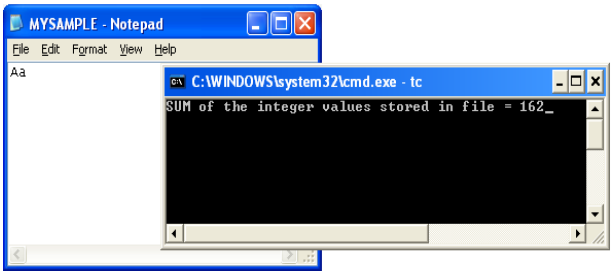
b)getw(*file_pointer):This function is used to read an integer value form the specified file which is opened in reading mode. If the data in file is set of characters then it reads ASCII values of those characters.

Example Program to illustrate getw() in C:

```
#include<stdio.h>
#include<conio.h>
```

```
void  main()
{
  FILE   *fp;
  int   i,j;
  clrscr();
  fp = fopen("MySample.txt","w");
  putw(65,fp);  // inserts A
  putw(97,fp);  // inserts a
  fclose(fp);
  fp = fopen("MySample.txt","r");
  i = getw(fp);   // reads 65 - ASCII value of A
  j = getw(fp);   // reads 97 - ASCII value of a
  printf("SUM of the integer values stored in file = %d", i+j);  // 65 + 97 = 162
  fclose(fp);
  getch();
}
```

Output:

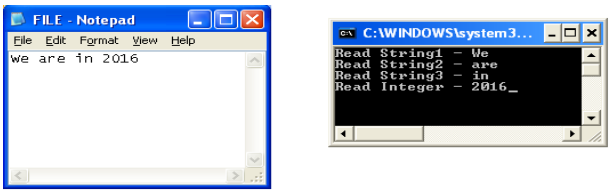


c)fscanf(*file_pointer, typeSpecifier, &variableName):This function is used to read multiple datatype values from specified file which is opened in reading mode.

Example Program to illustrate fscanf() in C:

```
#include<stdio.h>
#include<conio.h>
void  main()
{
  char str1[10], str2[10], str3[10];
  int year;
  FILE * fp;
  clrscr();
  fp = fopen ("file.txt", "w+");
  fputs("We are in 2016", fp);
  rewind(fp);   // moves the cursor to begining of the file
  fscanf(fp, "%s %s %s %d", str1, str2, str3, &year);
  printf("Read String1 - %s\n", str1 );
  printf("Read String2 - %s\n", str2 );
  printf("Read String3 - %s\n", str3 );
  printf("Read Integer - %d", year );
  fclose(fp);
  getch();
}
```

Output:

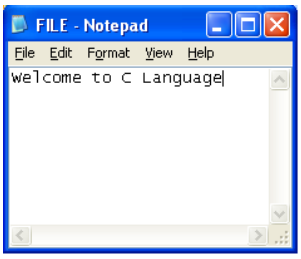


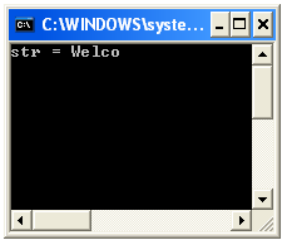
d)fgets(variableName, numberOfCharacters, *file_pointer):This method is used for reading a set of characters from a file which is opened in reading mode starting from the current cursor position. The fgets() function reading terminates with reading NULL character.

Example Program to illustrate fgets() in C:

```
#include<stdio.h>
#include<conio.h>
void  main()
{
  FILE   *fp;
  char   *str;
  clrscr();
  fp = fopen ("file.txt", "r");
  fgets(str,6,fp);
  printf("str = %s", str);
  fclose(fp);
  getch();
 }
```

Output:



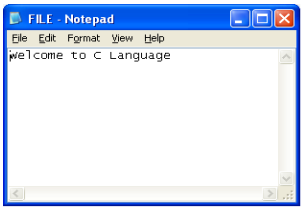


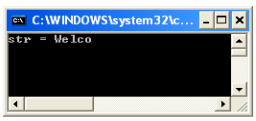
e)fread(source, sizeofReadingElement, numberOfCharacters, FILE *pointer):This function is used to read specific number of sequence of characters from the specified file which is opened in reading mode.

Example Program to illustrate fgets() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
  FILE *fp;
  char *str;
  clrscr();
  fp = fopen ("file.txt", "r");
  fread(str,sizeof(char),5,fp);
  str[strlen(str)+1] = 0;
  printf("str = %s", str);
  fclose(fp);
  getch();
}
```

Output:





3)Writing into a file:

The writing into a file operation is performed using the following pre-defined file handling methods.

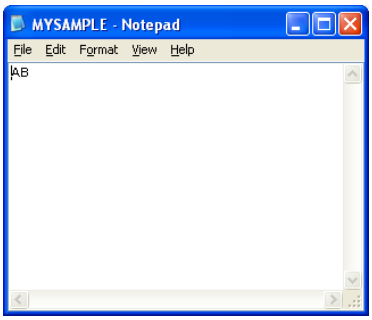
a) putc()
b) putw()
c) fprintf()
d) fputs()
e) fwrite()

a)putc(char, *file_pointer):This function is used to write/insert a character to the specified file when the file is opened in writing mode.

Example Program to illustrate putc() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
  FILE *fp;
  char ch;
  clrscr();
  fp = fopen("C:/TC/EXAMPLES/MySample.txt","w");
  putc('A',fp);
  ch = 'B';
  putc(ch,fp);
  fclose(fp);
  getch();
}
```

Output:

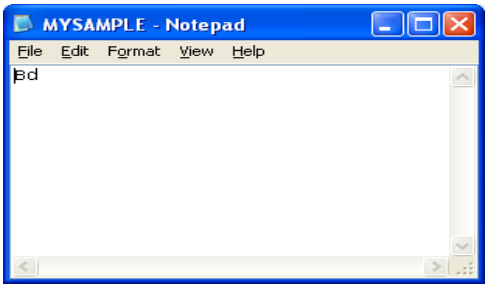


b)putw(int, *file_pointer):This function is used to writes/inserts an integer value to the specified file when the file is opened in writing mode.

Example Program to illustrate putw() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
  FILE *fp;
  int i;
  clrscr();
  fp = fopen("MySample.txt","w");
  putw(66,fp);
  i = 100;
  putw(i,fp);
  fclose(fp);
  getch();
}
```

Output:

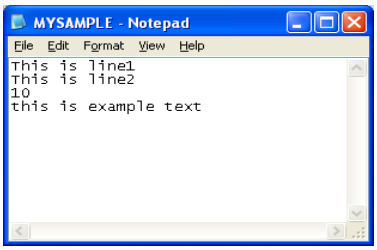


c)fprintf(*file_pointer, "text"):This function is used to writes/inserts multiple lines of text with mixed data types (char, int, float, double) into specified file which is opened in writing mode.

Example Program to illustrate "fprintf()" in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
 {
   FILE *fp;
   char *text = "\nthis is example text";
   int i = 10;
   clrscr();
   fp = fopen("MySample.txt","w");
   fprintf(fp,"This is line1\nThis is line2\n%d", i);
   fprintf(fp,text);
   fclose(fp);
   getch();
}
```

Output:

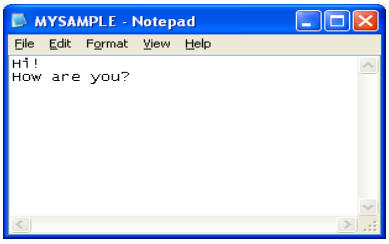


d)fputs("string", *file_pointer):TThis method is used to insert string data into specified file which is opened in writing mode.

Example Program to illustrate fputs() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
   FILE *fp;
   char *text = "\nthis is example text";
   clrscr();
   fp = fopen("MySample.txt","w");
   fputs("Hi!\nHow are you?",fp);
   fclose(fp);
   getch();
}
```

Output:



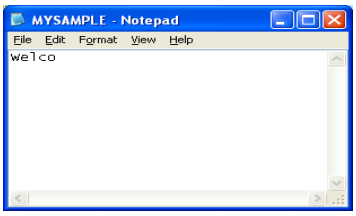
e)fwrite("StringData", sizeof(char), numberOfCharacters, FILE *pointer):This function is used to insert specified number of characters into a binary file which is opened in writing mode.

Example Program to illustrate fwrite() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
 {
```

```
 FILE *fp;
  char *text = "Welcome to C Language";
  clrscr();
  fp = fopen("MySample.txt","wb");
  fwrite(text,sizeof(char),5,fp);
  fclose(fp);
  getch();
}
```

Output:



4) Closing a file:

Closing a file is performed using a pre-defined method fclose().

fclose(*f_ptr)

The method fclose() returns '0'on success of file close otherwise it returns EOF (End Of File).

Random File Access in C(or)Cursor Positioning Functions in Files:

C programming language provides various pre-defined methods to set the cursor position in files. The following are the methods available in c, to position cursor in a file.

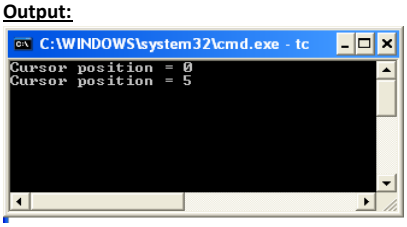
a) ftell()
b) rewind()
c) fseek()

a)ftell(*file_pointer):This function returns the current position of the cursor in the file.

Example Program to illustrate ftell() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
  FILE  *fp;
  int   position;
  clrscr();
  fp = fopen ("file.txt", "r");
  position = ftell(fp);
  printf("Cursor position = %d\n",position);
  fseek(fp,5,0);
  position = ftell(fp);
  printf("Cursor position = %d", position);
  fclose(fp);
  getch();
}
```

Output:



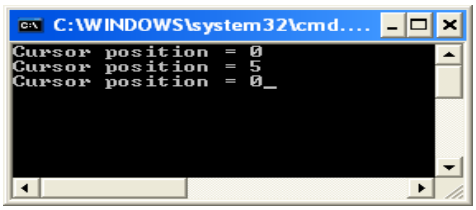
b)rewind(*file_pointer):This function is used reset the cursor position to the beginning of the file.

Example Program to illustrate rewind() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
```

```
{
  FILE *fp;
  int position;
  clrscr();
  fp = fopen ("file.txt", "r");
  position = ftell(fp);
  printf("Cursor position = %d\n",position);
  fseek(fp,5,0);
  position = ftell(fp);
  printf("Cursor position = %d\n", position);
  rewind(fp);
  position = ftell(fp);
  printf("Cursor position = %d", position);
  fclose(fp);
  getch();
}
```

Output:



c)fseek(*file_pointer, numberOfCharacters, fromPosition):This function is used to set the cursor position to the specific position. Using this function we can set the cursor position from three different position they are as follows.

- from beginning of the file (indicated with 0)
- from current cursor position (indicated with 1)
- from ending of the file (indicated with 2)

Example Program to illustrate fseek() in C:

```
#include<stdio.h>
#include<conio.h>
void   main()
{
  FILE *fp;
  int position;
  clrscr();
  fp = fopen ("file.txt", "r");
  position = ftell(fp);
  printf("Cursor position = %d\n",position);
  fseek(fp,5,0);
  position = ftell(fp);
  printf("Cursor position = %d\n", position);
  fseek(fp, -5, 2);
  position = ftell(fp);
  printf("Cursor position = %d", position);
  fclose(fp);
  getch();
}
```

Output:

